

Assignment - 2

Name: Divyansh Bhardwaj

Registration no. - RA2211026030052

Branch - CSE AI-ML

Year/Sem/Sec - 4th / 7th / 'A'

Date - 20/09/2025

Subject - Deep Learning

1. Discuss the limitations of plain gradient descent. Explain why optimization is difficult in deep neural networks.

Plain (or “vanilla”) gradient descent is the basic optimization algorithm in machine learning, but it struggles with the complexity of deep neural networks. In batch form, it computes gradients over the entire dataset, making it **slow and computationally expensive** for large-scale data. It also suffers from an **inability to escape local minima**, as neural network loss functions are non-convex. Moreover, it often encounters **saddle points**—flat regions where gradients approach zero—causing stagnation, and **ravines**, where steep, narrow valleys lead to oscillations and slow convergence.

Optimization in deep networks is especially difficult because their **loss landscapes are extremely high-dimensional and non-convex**, containing countless minima, saddle points, and plateaus. Additionally, deep architectures face **vanishing and exploding gradient problems** during backpropagation—gradients may either diminish to zero, halting learning, or blow up, causing instability. The **interdependence of parameters** further complicates updates, as changes to one weight affect many others.

In essence, training deep neural networks is like navigating a vast, rugged terrain rather than descending a simple slope. Plain gradient descent’s straightforward approach is often too inefficient and unstable to handle such complexity, leading to the need for more advanced optimization techniques like SGD, Momentum, RMSProp, and Adam.

2. Compare Momentum, Nesterov Accelerated Gradient, RMSProp, and Adam optimizers. Which one is generally preferred in practice and why?

These advanced optimization algorithms were designed to overcome the drawbacks of plain gradient descent by enhancing convergence speed and dynamically adjusting the learning process.

Comparison of Optimizers

1. Momentum

- **Main Idea:** Introduces a “velocity” component that accumulates an exponentially weighted average of previous gradients. This allows the optimizer to maintain consistent direction and accelerate movement through shallow regions or small local minima.
- **Analogy:** Like a heavy ball rolling downhill — it gathers momentum, smoothing out oscillations and quickly moving through valleys instead of getting stuck on minor bumps.

2. Nesterov Accelerated Gradient (NAG)

- **Main Idea:** An improvement over Momentum. Instead of computing the gradient at the current point, it first performs a small step in the direction of momentum (“lookahead”) and then calculates the gradient there. This helps anticipate the next slope and adjust the direction early, reducing the risk of overshooting.
- **Analogy:** Imagine a runner who looks ahead on the track to slow down before a sharp turn.

3. RMSProp (Root Mean Square Propagation)

- **Main Idea:** Uses adaptive learning rates by keeping an exponentially decaying average of squared gradients. The learning rate is divided by the square root of this value, allowing each parameter to adjust individually.
 - **Effect:** Parameters with larger gradients receive smaller updates, while those with smaller gradients receive larger ones, promoting balanced learning and smoother convergence in irregular terrains.
4. **Adam (Adaptive Moment Estimation)**
- **Main Idea:** Combines the principles of Momentum (first moment) and RMSProp (second moment). It tracks both the average of past gradients and their squared values, integrating acceleration with adaptive scaling.
 - **Effect:** Achieves both stability and speed, adapting to different parameters while retaining fast convergence.

Which Optimizer is Preferred and Why

Adam is the most commonly used optimizer in deep learning.

- **Hybrid Advantage:** It merges the strengths of Momentum and RMSProp, delivering efficient and adaptive updates.
- **Reliable Across Models:** Performs well across various architectures with minimal tuning.
- **Efficient:** Requires limited computational and memory resources, suitable for large-scale problems.
- **Fast Convergence:** Usually converges quickly and reliably, reducing training time.

Although **SGD with Momentum** can yield marginally better results with extensive tuning, **Adam** offers excellent performance by default, making it the preferred starting point for most deep learning tasks.

3. Explain the concept of bias-variance trade-off in deep learning. Use diagrams/examples to show underfitting vs. overfitting.

Concept of the Bias-Variance Trade-off

The bias-variance trade-off is a central concept in machine learning that deals with the challenge of building models that generalize well to new, unseen data. The total error of a model can be decomposed into three parts: $\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$. We focus on the first two.

- **Bias:** This is the error introduced by approximating a real-world problem, which may be complex, with a model that is too simple. A model with high bias makes strong assumptions about the data and fails to capture its underlying patterns. This leads to **underfitting**.
- **Variance:** This is the error introduced by a model that is too complex and overly sensitive to the small fluctuations (noise) in the training data. A model with high variance fits the training data perfectly but fails to generalize to new data. This leads to **overfitting**.

The Trade-off: There is an inherent inverse relationship. As we increase a model's complexity to reduce its bias, we increase its variance. Conversely, simplifying a model to reduce its variance will increase its bias. The goal is not to eliminate one or the other, but to find a "sweet spot" that minimizes the total error on unseen data.

Example: Fitting a Curve to Data Points

Imagine we have data points that follow a curved pattern.

- **Underfitting (High Bias):** If we try to model this data with a simple straight line (a low-complexity model), the line will be far from most points. It performs poorly on the training data and will also perform poorly on new data. This is underfitting.
- **Overfitting (High Variance):** If we use a very high-degree polynomial (a high-complexity model), we can draw a curve that passes through every single data point perfectly. However, this curve will be wildly erratic and will have learned the noise in the data, not the true underlying pattern. It will perform very poorly on new data. This is overfitting.
- **Good Fit (Balanced):** The best model is a smooth curve that captures the general trend of the data without trying to hit every single point. It has found the right balance between bias and variance.

4. Explain and compare L2 regularization, dataset augmentation, and early stopping as techniques to reduce overfitting in neural networks.

Overfitting occurs when a model with high variance learns the training data too well, including its noise, and fails to generalize to new data. L2 regularization, dataset augmentation, and early stopping are three of the most effective and widely used techniques to combat this.

1. L2 Regularization (Weight Decay)

- **Explanation:** L2 regularization is a technique that modifies the loss function by adding a penalty term. This penalty is proportional to the square of the magnitude of all the weights in the network.
New Loss = Original Loss + $\lambda \sum w^2$ where λ is the regularization hyperparameter that controls the strength of the penalty.
- **How it Reduces Overfitting:** By penalizing large weights, L2 regularization encourages the model to learn smaller, more distributed weight values. A network with smaller weights is less complex and produces a smoother decision boundary, making it less sensitive to noise in the training data and more likely to generalize well.

2. Dataset Augmentation

- **Explanation:** This technique involves artificially increasing the size and diversity of the training dataset by creating modified versions of the existing data. For image data, common augmentations include random rotations, flips, zooms, crops, and color adjustments.
- **How it Reduces Overfitting:** Overfitting often happens when the model has too little data to learn from. By providing the model with more and varied examples, augmentation makes it harder for the model to simply memorize the training set. It learns to be invariant to transformations that don't change the object's identity (e.g., a cat is still a cat even if the image is flipped), leading to a more robust and generalizable model.

3. Early Stopping

- **Explanation:** Early stopping is a technique that modifies the training process itself. During training, the model's performance (e.g., loss or accuracy) is monitored on a separate validation dataset. The training process is halted as soon as the performance on the validation set stops improving and begins to degrade, even if the training loss is still decreasing.

- **How it Reduces Overfitting:** The point where the validation loss begins to increase is the point where the model starts to overfit the training data. By stopping training at this optimal moment, we prevent the model from learning the noise and save the model weights that generalize best.

Comparison of the Techniques

Feature	L2 Regularization	Dataset Augmentation	Early Stopping
Method	Modifies the model's objective (loss function).	Modifies the data .	Modifies the training procedure .
Core Principle	Penalize model complexity.	Increase data diversity and quantity.	Stop training at the optimal point of generalization.
Computational Cost	Very low overhead.	Can be computationally intensive, especially for complex augmentations.	Very low overhead; can reduce overall training time.
Primary Use Case	A standard, general-purpose technique for almost any network.	Extremely effective for computer vision; useful whenever data is limited.	A simple and highly effective technique for virtually any iterative training process.

These three techniques are not mutually exclusive and are often used in combination to achieve the best results in preventing overfitting.

5. Discuss the relation between Principal Component Analysis (PCA) and Autoencoders. How do they achieve dimensionality reduction differently?

Introduction

Both Principal Component Analysis (PCA) and Autoencoders are powerful techniques for dimensionality reduction. They share the same goal of representing data in a lower-dimensional space while preserving essential information, but they achieve this goal through fundamentally different approaches.

How They Achieve Dimensionality Reduction Differently

Principal Component Analysis (PCA)

- **Method:** PCA is a **linear**, mathematical algorithm. It does not involve any "training" in the neural network sense.

- **Process:** It analyzes the covariance matrix of the data to find a new set of orthogonal (uncorrelated) axes, called principal components. These components are ordered such that the first component captures the maximum possible variance in the data, the second captures the next most, and so on. Dimensionality is reduced by keeping only the first few principal components and discarding the rest.
- **Transformation:** The transformation is a **linear projection** of the data onto the subspace defined by the top principal components.

Autoencoders

- **Method:** An autoencoder is a type of **neural network** that is trained through backpropagation. It is a learning-based approach.
- **Process:** It consists of two sub-networks:
 1. An **Encoder** that learns to compress (encode) the high-dimensional input data into a low-dimensional latent space representation (the "bottleneck").
 2. A **Decoder** that learns to reconstruct the original data from this compressed representation.
-
- The network is trained to minimize the **reconstruction error**—the difference between the original input and the final output. This forces the encoder to learn a meaningful compression that captures the most salient features of the data.
- **Transformation:** The transformation is learned by the encoder and can be highly **non-linear** if non-linear activation functions are used.

The Relation Between PCA and Autoencoders

The relationship between these two methods is deep and reveals that autoencoders can be seen as a powerful, non-linear generalization of PCA.

The key connection is this: **A simple autoencoder with a single hidden layer, linear activation functions, and a mean squared error loss function will learn to perform the exact same dimensionality reduction as PCA.** The hidden layer of this linear autoencoder will learn to span the same subspace as the one defined by the first k principal components of the data, where k is the number of neurons in the hidden layer.

Therefore:

- **PCA is a special case of an autoencoder.**
- By adding multiple hidden layers and, most importantly, **non-linear activation functions (like ReLU or Sigmoid)**, autoencoders can move beyond PCA's linear limitations. They can learn to reduce the dimensionality of data that lies on complex, curved manifolds—something PCA is incapable of doing effectively.

Summary of Differences

Aspect	Principal Component Analysis (PCA)	Autoencoder
Nature of Transformation	Strictly Linear	Can be Non-Linear
Methodology	Mathematical Algorithm (Eigendecomposition)	Learning-Based (Neural Network Training)
Flexibility	Fixed algorithm	Highly flexible architecture
Computational Cost	Fast, deterministic	Slower to train, iterative
Power	Captures directions of maximum variance	Captures salient features to minimize reconstruction error, can model complex patterns